

Token Classification

The first application we will explore is **token classification**.

This task means assigning a **label to each token (word)** in a sentence.

Examples of Token Classification

1. Named Entity Recognition (NER)

This task finds entities in a sentence, such as:

- Person
- Location
- Organization

For each token:

- If it is an entity → assign its type
- If not → assign “no entity”

2. Part-of-Speech Tagging (POS)

This task assigns a grammatical role to each word, such as:

- Noun
- Verb
- Adjective

3. Chunking

This task groups tokens that belong to the same entity.

Labels used:

- **B-** → beginning of a chunk
- **I-** → inside a chunk
- **O** → outside any chunk

There are many other token classification problems, but these are the most common ones.

In this section, we will **fine-tune a BERT model for NER**.

Preparing the Data

First, we need a dataset.

Here, we use the **CoNLL-2003 dataset**, which is built from Reuters news data.

Important Note

If your dataset has:

- Text → list of words
- Labels → label for each word

Then you can apply this same method to your own dataset.

Load Dataset

Code

```
from datasets import load_dataset

raw_datasets = load_dataset("conll2003")
```

Explanation

- Downloads the dataset
- Saves it in cache

Dataset Structure

Code

```
raw_datasets
```

Output

```
DatasetDict({
  train: Dataset({
    features: ['chunk_tags', 'id', 'ner_tags', 'pos_tags', 'tokens'],
    num_rows: 14041
  })
  validation: Dataset({
    features: ['chunk_tags', 'id', 'ner_tags', 'pos_tags', 'tokens'],
    num_rows: 3250
  })
  test: Dataset({
    features: ['chunk_tags', 'id', 'ner_tags', 'pos_tags', 'tokens'],
    num_rows: 3453
  })
})
```

Explanation

The dataset is divided into:

- Train
- Validation
- Test

Each part has 5 columns:

- **tokens** → list of words
- **ner_tags** → NER labels
- **pos_tags** → POS labels
- **chunk_tags** → chunk labels
- **id** → sample ID

Important Difference

Here, text is NOT given as full sentences.

Instead, it is given as a **list of words (tokens)**.

Example Data

Code

```
raw_datasets["train"][0]["tokens"]
```

Output

```
['EU', 'rejects', 'German', 'call', 'to', 'boycott', 'British', 'lamb', '.']
```

Explanation

This is a sentence, but represented as a list of words.

NER Labels

Code

```
raw_datasets["train"][0]["ner_tags"]
```

Output

```
[3, 0, 7, 0, 0, 0, 7, 0, 0]
```

Explanation

- These are integer labels
- Used for training
- Hard to understand directly

Get Label Names

Code

```
ner_feature = raw_datasets["train"].features["ner_tags"]
```

ner_feature

Output

```
Sequence(feature=ClassLabel(num_classes=9, names=['O', 'B-PER', 'I-PER', 'B-ORG', 'I-ORG', 'B-LOC', 'I-LOC', 'B-MISC', 'I-MISC'], names_file=None, id=None), length=-1, id=None)
```

Label Names List

Code

```
label_names = ner_feature.feature.names  
label_names
```

Output

```
['O', 'B-PER', 'I-PER', 'B-ORG', 'I-ORG', 'B-LOC', 'I-LOC', 'B-MISC', 'I-MISC']
```

Label Meaning

- **O** → Not an entity
- **B-PER / I-PER** → Person
- **B-ORG / I-ORG** → Organization
- **B-LOC / I-LOC** → Location
- **B-MISC / I-MISC** → Other entities

Decode Labels

Code

```
words = raw_datasets["train"][0]["tokens"]  
labels = raw_datasets["train"][0]["ner_tags"]
```

```
line1 = ""  
line2 = ""
```

```

for word, label in zip(words, labels):
    full_label = label_names[label]
    max_length = max(len(word), len(full_label))
    line1 += word + " " * (max_length - len(word) + 1)
    line2 += full_label + " " * (max_length - len(full_label) + 1)

print(line1)
print(line2)

```

Output

```

'EU   rejects German call to boycott British lamb .'
'B-ORG O   B-MISC O   O O   B-MISC O   O'

```

Explanation

- EU → B-ORG (organization)
- German → B-MISC
- British → B-MISC

Some words are labeled as entities.

B- and I- Label Example

Example

'Germany \s representative to the European Union \s veterinary committee Werner Zwingmann said on Wednesday consumers should buy sheepmeat from countries other than Britain until the scientific advice was clearer .'

```

'B-LOC O O           O O B-ORG I-ORG O O           O   B-PER I-PER O   O O
O   O   O O   O O   O   O B-LOC O   O O   O   O O O   O'

```

Explanation

Important examples:

- **European Union**
 - European → B-ORG
 - Union → I-ORG

- **Werner Zwingmann**
 - Werner → B-PER
 - Zwingmann → I-PER

Key Idea

- **B-** → beginning of entity
- **I-** → inside of entity

Processing the Data

As before, we need to convert our text into **token IDs** so that the model can understand it.

However, in token classification there is one important difference:

- The input is already split into words (pre-tokenized)

The good news is that the tokenizer API can handle this easily.

We just need to use a **special flag**.

Creating the Tokenizer

Code

```
from transformers import AutoTokenizer
```

```
model_checkpoint = "bert-base-cased"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
```

Explanation

- We are using a **pretrained BERT tokenizer**
- It will be downloaded from the Hugging Face Hub

Check Fast Tokenizer

Code

```
tokenizer.is_fast
```

Output

```
True
```

Explanation

- **True** means this is a **fast tokenizer**
- Fast tokenizers provide extra features like `word_ids()`

Tokenizing Pre-tokenized Input

Code

```
inputs = tokenizer(raw_datasets["train"][0]["tokens"], is_split_into_words=True)
inputs.tokens()
```

Output

```
[['[CLS]', 'EU', 'rejects', 'German', 'call', 'to', 'boycott', 'British', 'la', '##mb', '.', '[SEP]']]
```

Explanation

- **[CLS]** → start of sentence
- **[SEP]** → end of sentence
- Most words remain the same
- But **"lamb"** is split into subwords:
 - **la**

- ##mb

Problem: Label Mismatch

- Original labels → 9
- Tokenized input → 12

This happens because:

- Special tokens are added
- Some words are split into subwords

So we need to **adjust the labels**.

Token to Word Mapping

Code

```
inputs.word_ids()
```

Output

```
[None, 0, 1, 2, 3, 4, 5, 6, 7, 7, 8, None]
```

Explanation

- Shows which word each token comes from
- **None** → special tokens
- **7, 7** → same word split into two tokens (**1amb**)

Function to Align Labels

Code

```
def align_labels_with_tokens(labels, word_ids):  
    new_labels = []  
    current_word = None
```

```

for word_id in word_ids:
    if word_id != current_word:
        # Start of a new word!
        current_word = word_id
        label = -100 if word_id is None else labels[word_id]
        new_labels.append(label)
    elif word_id is None:
        # Special token
        new_labels.append(-100)
    else:
        # Same word as previous token
        label = labels[word_id]
        # If the label is B-XXX we change it to I-XXX
        if label % 2 == 1:
            label += 1
        new_labels.append(label)

return new_labels

```

Simple Explanation of the Function

This function does three things:

1. Special tokens → assign **-100**
2. All tokens of the same word → get the same label
3. If label is **B-XXX**, then inner tokens become **I-XXX**

Test the Function

Code

```

labels = raw_datasets["train"][0]["ner_tags"]
word_ids = inputs.word_ids()

print(labels)
print(align_labels_with_tokens(labels, word_ids))

```

Output

[3, 0, 7, 0, 0, 0, 7, 0, 0]
[-100, 3, 0, 7, 0, 0, 0, 7, 0, 0, 0, -100]

Explanation

- **-100** is added for special tokens
- Extra label added because "lamb" was split
- During training, **-100** will be ignored

Alternative Idea

Some researchers prefer:

- Only assign label to the **first token of each word**
- Assign **-100** to the rest

This helps because:

- Long words do not dominate the loss

Full Dataset Preprocessing

Now we process the entire dataset:

- Tokenize inputs
- Align labels

Create Function

Code

```
def tokenize_and_align_labels(examples):  
    tokenized_inputs = tokenizer(  
        examples["tokens"], truncation=True, is_split_into_words=True  
    )  
    all_labels = examples["ner_tags"]  
    new_labels = []  
    for i, labels in enumerate(all_labels):
```

```
word_ids = tokenized_inputs.word_ids(i)
new_labels.append(aligned_labels_with_tokens(labels, word_ids))

tokenized_inputs["labels"] = new_labels
return tokenized_inputs
```

Explanation

- Processes multiple samples at once
- Aligns labels for each example
- Adds labels to tokenized output

Apply to Dataset

Code

```
tokenized_datasets = raw_datasets.map(
    tokenize_and_align_labels,
    batched=True,
    remove_columns=raw_datasets["train"].column_names,
)
```

Explanation

- Processes the whole dataset
- `batched=True` → faster processing
- Removes old columns

Fine-tuning the Model with the Trainer API

The training code using the **Trainer API** is almost the same as before.

Only two things are different:

1. Data collation (how batches are created)
2. Metric computation

Data Collation

We cannot use `DataCollatorWithPadding` like in Chapter 3.

Because:

- It only pads inputs (`input_ids`, `attention_mask`, `token_type_ids`)
- But here, we also need to pad **labels**

Also:

- Labels must be padded with **-100**
- So they are ignored during loss calculation

Using DataCollatorForTokenClassification

Code

```
from transformers import DataCollatorForTokenClassification

data_collator = DataCollatorForTokenClassification(tokenizer=tokenizer)
```

Explanation

- Pads both inputs and labels
- Uses `-100` for label padding

Testing the Data Collator

Code

```
batch = data_collator([tokenized_datasets["train"][i] for i in range(2)])  
batch["labels"]
```

Output

```
tensor([[ -100,  3,  0,  7,  0,  0,  0,  7,  0,  0,  0, -100],  
        [ -100,  1,  2, -100, -100, -100, -100, -100, -100, -100, -100, -100]])
```

Explanation

- Two samples are combined into one batch
- The shorter label list is padded with **-100**

Compare with Original Labels

Code

```
for i in range(2):  
    print(tokenized_datasets["train"][i]["labels"])
```

Output

```
[-100, 3, 0, 7, 0, 0, 0, 7, 0, 0, 0, -100]  
[-100, 1, 2, -100]
```

Explanation

- The second sample was shorter

- It is padded with `-100` in the batch

Metrics

To compute metrics after each epoch, we need a `compute_metrics()` function.

Install sequeval

Code

```
!pip install sequeval
```

Load Metric

Code

```
import evaluate

metric = evaluate.load("sequeval")
```

Decode Labels

Code

```
labels = raw_datasets["train"][0]["ner_tags"]
labels = [label_names[i] for i in labels]
labels
```

Output

```
['B-ORG', 'O', 'B-MISC', 'O', 'O', 'O', 'B-MISC', 'O', 'O']
```

Explanation

- Integer labels are converted into string labels

Create Fake Predictions

Code

```
predictions = labels.copy()
predictions[2] = "O"
```

```
metric.compute(predictions=[predictions], references=[labels])
```

Output

```
{'MISC': {'precision': 1.0, 'recall': 0.5, 'f1': 0.67, 'number': 2},
'ORG': {'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'number': 1},
'overall_precision': 1.0,
'overall_recall': 0.67,
'overall_f1': 0.8,
'overall_accuracy': 0.89}
```

Explanation

For each entity, we get:

- Precision
- Recall
- F1 score

Also overall scores are returned.

compute_metrics Function

Code

```
import numpy as np
```

```
def compute_metrics(eval_preds):
```

```
    logits, labels = eval_preds
```

```
    predictions = np.argmax(logits, axis=-1)
```

```
    # Remove ignored index (special tokens) and convert to labels
```

```
    true_labels = [[label_names[l] for l in label if l != -100] for label in labels]
```

```
    true_predictions = [
```

```
        [label_names[p] for (p, l) in zip(prediction, label) if l != -100]
```

```
        for prediction, label in zip(predictions, labels)
```

```
    ]
```

```
    all_metrics = metric.compute(predictions=true_predictions, references=true_labels)
```

```
    return {
```

```
        "precision": all_metrics["overall_precision"],
```

```
        "recall": all_metrics["overall_recall"],
```

```
        "f1": all_metrics["overall_f1"],
```

```
        "accuracy": all_metrics["overall_accuracy"],
```

```
    }
```

Explanation

This function:

1. Converts logits to predictions using `argmax`
2. Removes `-100` labels
3. Converts integer labels to strings
4. Computes metrics

Defining the Model

For token classification, we use:

Code

```
id2label = {i: label for i, label in enumerate(label_names)}  
label2id = {v: k for k, v in id2label.items()}
```

Load Model

Code

```
from transformers import AutoModelForTokenClassification  
  
model = AutoModelForTokenClassification.from_pretrained(  
    model_checkpoint,  
    id2label=id2label,  
    label2id=label2id,  
)
```

Explanation

- Loads pretrained BERT model
- Adds a new classification head

Check Number of Labels

Code

```
model.config.num_labels
```

Output

9

Explanation

- Dataset has 9 labels
- Model matches correctly

Before Training

Hugging Face Login

```
from huggingface_hub import notebook_login
```

```
notebook_login()
```

Training Arguments

Code

```
from transformers import TrainingArguments
```

```
args = TrainingArguments(  
    "bert-finetuned-ner",  
    evaluation_strategy="epoch",  
    save_strategy="epoch",  
    learning_rate=2e-5,  
    num_train_epochs=3,  
    weight_decay=0.01,  
    push_to_hub=True,  
)
```

Explanation

- learning_rate → training speed
- epochs → number of training cycles
- weight_decay → regularization
- push_to_hub → upload model

Create Trainer

Code

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    data_collator=data_collator,
    compute_metrics=compute_metrics,
    processing_class=tokenizer,
)
```

Start Training

Code

```
trainer.train()
```

Explanation

- Training starts
- Evaluation runs after each epoch
- Model is uploaded to the Hub

Upload Final Model

Code

```
trainer.push_to_hub(commit_message="Training complete")
```

Output

'https://huggingface.co/sgugger/bert-finetuned-ner/commit/26ab21e5b1568f9afeccdaed2d8715f571d786ed'

Explanation

- Model is uploaded to Hugging Face Hub
- You can check it using the link

Final Result

- Trainer automatically creates a **model card**
- Saves evaluation results
- Provides an **inference widget** to test the model

A Custom Training Loop

Now we will create a **fully custom training loop**, so you can customize everything yourself. It is similar to Chapter 3, but the **evaluation part** has some differences.

Preparing Everything for Training

We first need to create **DataLoaders** from the dataset.

DataLoader Creation

```
from torch.utils.data import DataLoader
```

```
train_dataloader = DataLoader(  
    tokenized_datasets["train"],  
    shuffle=True,  
    collate_fn=data_collator,  
    batch_size=8,  
)
```

```
eval_dataloader = DataLoader(  
    tokenized_datasets["validation"],  
    collate_fn=data_collator,  
    batch_size=8  
)
```

Explanation:

- `train_dataloader` → training data (shuffled)
- `eval_dataloader` → validation data (not shuffled)
- `batch_size=8` → 8 samples per batch

Model Reload

```
model = AutoModelForTokenClassification.from_pretrained(  
    model_checkpoint,  
    id2label=id2label,
```

```
label2id=label2id,  
)
```

Explanation:

- Load a fresh model
- Not continuing from previous training

Optimizer

```
from torch.optim import AdamW
```

```
optimizer = AdamW(model.parameters(), lr=2e-5)
```

Explanation:

- Using **AdamW optimizer**
- Learning rate = $2e-5$

Accelerator for GPU/TPU

```
from accelerate import Accelerator
```

```
accelerator = Accelerator()  
model, optimizer, train_dataloader, eval_dataloader = accelerator.prepare(  
    model, optimizer, train_dataloader, eval_dataloader  
)
```

Explanation:

- Prepares model, optimizer, and data for GPU/TPU
- Simplifies distributed training

Learning Rate Scheduler

```
from transformers import get_scheduler
```

```
num_train_epochs = 3
num_update_steps_per_epoch = len(train_dataloader)
num_training_steps = num_train_epochs * num_update_steps_per_epoch

lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps,
)
```

Explanation:

- Learning rate **linearly decreases** during training
- Total training steps are computed

Hugging Face Repository Setup

```
from huggingface_hub import Repository, get_full_repo_name

model_name = "bert-finetuned-ner-accelerate"
repo_name = get_full_repo_name(model_name)
repo_name
```

Output:

```
'sgugger/bert-finetuned-ner-accelerate'
```

Explanation:

- Repository created to **upload model**

Clone Repository

```
output_dir = "bert-finetuned-ner-accelerate"
repo = Repository(output_dir, clone_from=repo_name)
```


Postprocess Function

```
def postprocess(predictions, labels):
    predictions = predictions.detach().cpu().clone().numpy()
    labels = labels.detach().cpu().clone().numpy()

    # Remove ignored index (-100) and convert to string labels
    true_labels = [[label_names[l] for l in label if l != -100] for label in labels]
    true_predictions = [
        [label_names[p] for (p, l) in zip(prediction, label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]
    return true_labels, true_predictions
```

Explanation:

- Converts **tensors** → **numpy**
- Removes **-100** special tokens
- Converts integer labels → string labels

Full Training Loop

```
from tqdm.auto import tqdm
import torch

progress_bar = tqdm(range(num_training_steps))

for epoch in range(num_train_epochs):
    # ----- Training -----
    model.train()
    for batch in train_dataloader:
        outputs = model(**batch)
        loss = outputs.loss
        accelerator.backward(loss)

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)

    # ----- Evaluation -----
    model.eval()
```

```

for batch in eval_dataloader:
    with torch.no_grad():
        outputs = model(**batch)

    predictions = outputs.logits.argmax(dim=-1)
    labels = batch["labels"]

    # Pad predictions and labels for gathering across processes
    predictions = accelerator.pad_across_processes(predictions, dim=1, pad_index=-100)
    labels = accelerator.pad_across_processes(labels, dim=1, pad_index=-100)

    predictions_gathered = accelerator.gather(predictions)
    labels_gathered = accelerator.gather(labels)

    true_predictions, true_labels = postprocess(predictions_gathered, labels_gathered)
    metric.add_batch(predictions=true_predictions, references=true_labels)

results = metric.compute()
print(
    f"epoch {epoch}:",
    {key: results[f"overall_{key}"] for key in ["precision", "recall", "f1", "accuracy"]},
)

# ----- Save & Upload -----
accelerator.wait_for_everyone()
unwrapped_model = accelerator.unwrap_model(model)
unwrapped_model.save_pretrained(output_dir, save_function=accelerator.save)
if accelerator.is_main_process:
    tokenizer.save_pretrained(output_dir)
    repo.push_to_hub(
        commit_message=f"Training in progress epoch {epoch}", blocking=False
    )

```

Explanation (Simple)

This loop does three things:

1. **Training**
 - Sets model to **train mode**
 - Forward pass → compute loss
 - Backward pass → update gradients
2. **Evaluation**
 - Sets model to **eval mode**

- Generates predictions
 - Pads predictions & labels across processes
 - Computes metrics
3. **Save & Upload**
- Waits for all processes
 - Unwraps model and saves
 - Pushes model to **Hugging Face Hub**

Model Save Code Explained

```
accelerator.wait_for_everyone()
unwrapped_model = accelerator.unwrap_model(model)
unwrapped_model.save_pretrained(output_dir, save_function=accelerator.save)
```

- Waits for **all processes**
- **Unwraps** model from accelerator
- Saves to disk

Using the Fine-tuned Model

```
from transformers import pipeline
```

```
# Replace this with your own checkpoint
model_checkpoint = "huggingface-course/bert-finetuned-ner"
```

```
token_classifier = pipeline(
    "token-classification", model=model_checkpoint, aggregation_strategy="simple"
)
```

```
token_classifier("My name is Sylvain and I work at Hugging Face in Brooklyn.")
```

Output:

```
[
  {'entity_group': 'PER', 'score': 0.9988506, 'word': 'Sylvain', 'start': 11, 'end': 18},
  {'entity_group': 'ORG', 'score': 0.9647625, 'word': 'Hugging Face', 'start': 33, 'end': 45},
  {'entity_group': 'LOC', 'score': 0.9986118, 'word': 'Brooklyn', 'start': 49, 'end': 57}
]
```

Explanation:

- Sylvain → Person
- Hugging Face → Organization
- Brooklyn → Location